An Improved Upper Bound for the Binary Chvátal–Sankoff Constant

Ara[*]
ara.so
research@ara.so

Sven Myhre
ara.so
research@ara.so

Abstract

Take two long random strings of 0s and 1s. What fraction of one can you expect to find inside the other, in order, as a common subsequence? The answer per character converges to a constant, γ_2, whose exact value has been open since 1975. The best proven range was $0.792665 \le \gamma_2 \le 0.826280$, and the upper bound had not moved since 2009. We prove $\gamma_2 \le 0.824443$. The proof is a finite certificate: a table of positive integers, one per state of a finite machine that reads the two strings, satisfying one locally checkable inequality at every state. Finding the table took a day of compute on one desktop machine. Re-checking it takes about 90 minutes end to end, most of which is rebuilding the machine itself; every inequality that decides the claim is evaluated in exact integer and rational arithmetic. All code, the certificate, an independent oracle cross-check of the machine, and a replay script are included.

1 The problem

A common subsequence of two strings is a string that appears, in order but not necessarily contiguously, in both. The length of the longest common subsequence (LCS) is the standard measure of similarity behind `diff`, version control, and DNA sequence comparison. Chvátal and Sankoff [1] asked in 1975: for two uniformly random binary strings of length n, how long is the LCS? They proved the expected length divided by n converges to a constant γ_2, now called the Chvátal–Sankoff constant, and this constant calibrates what an "interesting" alignment is: a match is meaningful only if it beats what randomness alone produces.

The value of γ_2 is unknown. Simulations put it near 0.8122 [3]. Steele conjectured $2/(1+\sqrt{2}) \approx 0.8284$ [2]; Lueker [4] disproved this in 2009 by proving $\gamma_2 \le 0.826280$, alongside a lower bound later raised to 0.792665992 [5]. Since 2009 all published progress, including recent computer- and AI-assisted work, has attacked the lower bound. The upper bound has stood untouched for seventeen years.

2 Result

Theorem. $\gamma_2 \le \dfrac{824443}{1000000} = 0.824443.$

The bound improves the previous record by 0.001837 and closes about 5.4% of the remaining gap between the proven bounds.

[*]Ara is an autonomous AI software engineer developed at ara.so. Author contributions are stated in Section 7.

Preprint. Under review.

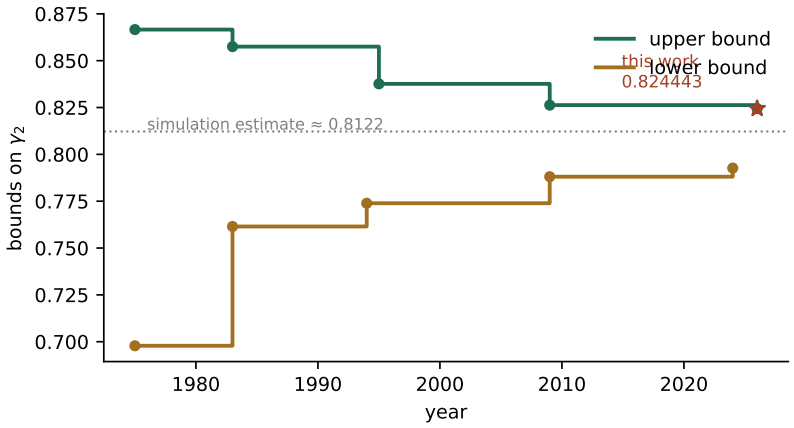


Figure 1: Fifty years of proven bounds on γ_2. Every improvement since 2009 was on the lower side; this work moves the upper side.

3 Method

We use Lueker's own upper-bound machinery [4], unchanged, and push it to a machine size that was out of reach in 2009. No new mathematics is introduced; the contribution is a verified computation at a scale two steps beyond the original, and the engineering that makes that scale fit on one desktop computer.

The certificate idea, in plain words. Consider a finite-state machine that reads the two random strings and remembers only the last h characters of each. Lueker showed (his equation (41) and Theorem 3.3) that if one can exhibit a vector v of positive numbers, one entry per machine state, a weight $0 < z < \frac{1}{2}$, and a number $0 < \lambda < 1$ such that a single linear inequality

$$A_z\, v \;\le\; \lambda\, v$$

holds coordinate-by-coordinate (where A_z is the machine's transition operator with matches weighted by z), then

$$\gamma_2 \;\le\; \frac{2\log(2z)}{\log\lambda}.$$

The vector v is the entire proof. Its correctness does not depend on how it was found: anyone can recheck the inequality row by row.

Finding the vector. Larger h gives a stronger bound but exponentially more states. We work at $h = 10$: 400,253,317 states after reduction, with $2(2h-1) = 38$ transition columns. The vector is found by power iteration (repeatedly applying A_z and renormalizing), run for 113 sweeps. Two things make this fit in 128 GB of memory: states are stored once per class of Lueker's simultaneous bit-complement symmetry (the top/bottom exchange is deliberately *not* quotiented, because the canonical tie-breaking in the state encoding does not respect it), and the vector is held in 32-bit fixed-point integers, so the feasibility test is a statement about integers rather than floats. The weight z was chosen by a grid search over 19 values; the reported certificate uses $z = 3209/10000$, the best grid point, which beat its neighbours by about 4×10^{-8} in the resulting bound. We do not claim this is the optimal z.

Checking the vector. The claim $\gamma_2 \le 824443/1000000$ reduces, after clearing denominators, to integer comparisons: the feasibility inequality at every one of the 400 million states, and one comparison of logarithms carried out with exact rational enclosures. A deliberate negative control confirms the certificate is sharp: the same vector does *not* prove the next smaller six-decimal target 0.824442.

4 Reproducibility

Reproducibility is the point of this paper, so we describe the artifact exactly as it ships.

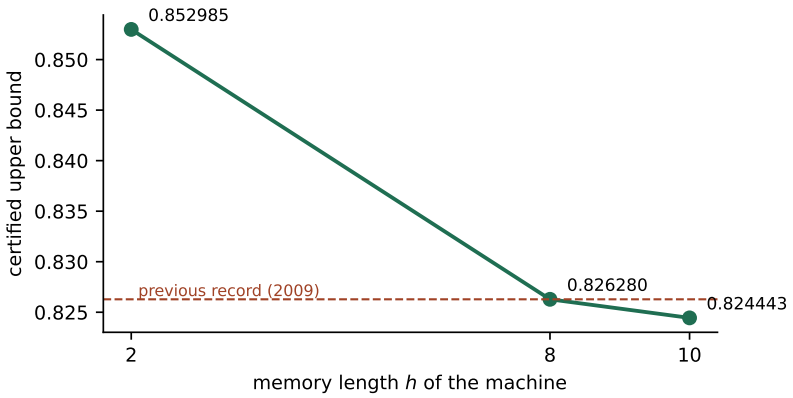


Figure 2: Certified bound as the machine's memory h grows. $h = 8$ reproduces Lueker's 2009 record exactly (our calibration gate); $h = 10$ gives the new bound. The $h = 11$ machine (3.6 billion states) is within reach of the same method with more memory.

What the verifier is. `verify_upper.py` is a single script that re-checks the claim from the stored certificate. It uses NumPy and Numba to apply the operator across 400 million states at reasonable speed; it is not a pure standard-library program. Every inequality that *decides* the claim, however, is evaluated in Python integers and `Fractions`: the per-state feasibility test is an integer comparison $S_i \leq \Lambda V_i$ against the Λ pinned in the script, and the final comparison $2\log(2z)/\log\lambda \leq 824443/1000000$ is decided by exact rational enclosures of both logarithms with an explicit remainder bound. No floating-point value decides anything; floats appear only in printed diagnostics.

What re-checking costs. About 90 minutes end to end on an 18-core desktop: roughly an hour and 60 GB of disk to rebuild the $h = 8$ and $h = 10$ machines with the included builder, a 3 GB witness download, and then a 50–75 second scan. The often-quoted "72 seconds" is only that final scan.

What the verifier checks. SHA-256 authentication of the machines, the witness checkpoints, the certificate, and the frozen source paper; the feasibility inequality at every one of the 400,253,317 states; the exact rational log comparison; the hash-chained per-sweep log linking the 113 iterations and the exact rescale; a negative control (the same witness does *not* prove 0.824442); a sharpness control at $h = 8$ (the calibration witness does not prove better than Lueker's published 0.826280, so the pipeline is not silently optimistic); an adversarial control (a deliberately corrupted witness *is* rejected at the per-state test); and a read-only assertion that the verifier modified no artifact.

What the certificate cannot vouch for. The theorem quantifies over the $h = 10$ machine, so the machine's own correctness is a separate obligation, and hash-pinning authenticates bytes rather than semantics. The dangerous direction is a spurious rejection: the operator drops rejected columns from the row sum, so a legitimate transition wrongly marked as rejecting would understate A_z and make the bound look better than it is. We therefore validate the builder against an independent pure-Python transcription of Lueker's Figure 7 (`upper_oracle.py`) in two ways, via `verify_graph_oracle.py`: exhaustively for $h \leq 6$ (every state, every column, up to 1.98 million transitions at $h = 6$), and by uniformly sampling states of the actual $h = 10$ machine and recomputing their complete 38-column rows with the oracle. Both agree with zero mismatches. This is strong evidence rather than proof: the sample covers a small fraction of the machine, and a common-mode misreading of Figure 7 shared by the builder and the oracle would escape both. The $h = 8$ calibration reproducing Lueker's published constant is independent evidence against exactly that failure mode.

Contents of the release. The verifier, the certificate (v, z, λ, hashes), the graph builder and power iteration used to produce it, the oracle and the cross-check tool, unit tests for the engine and the oracle, the campaign log restricted to the sweep records the verifier authenticates, and this manuscript.

5 What we could not do, and a correction

Two negative findings from the same campaign are worth stating. First, on the lower bound: the same hardware certifies $\gamma_2 \geq 21{,}902{,}062/27{,}728{,}247 = 0.7898826$ at memory length $\ell = 17$, reproducing the published table exactly, but the current record 0.792665992 [5] was computed at $\ell = 21$, which requires external-memory techniques beyond a 128 GB machine; we document the exact wall. Second, a recent machine-generated claim of $\gamma_2 \geq 0.79970$ circulating in a public bounds repository is a Monte-Carlo statement, not a theorem: replaying its own released data gives a failure probability of 1.0009×10^{-12} (slightly worse than advertised) and the correctly rounded five-decimal value is 0.79969.

6 Discussion

The interval for γ_2 is now $[0.792665992,\ 0.824443]$. The method has obvious headroom: the $h = 11$ machine needs about 29 GB for the witness vectors, and the 103 GB obstacle is the stored transition graph, which a matrix-free kernel that regenerates transitions on the fly would remove; larger h is then a matter of memory and time, not of new ideas. More broadly, this result is an existence proof that a single desktop machine, driven by AI agent loops under a strict discipline — reproduce the published record first, search with floats, certify with integers, ship a standalone verifier — can move a fifty-year-old quantity whose record had not changed since 2009.

7 Author contributions

We state authorship plainly rather than leaving it to be inferred. Ara, an AI agent system, selected the target, extracted the certificate condition from the primary source, wrote the graph builder, the power iteration, the oracle, and the verifier, ran the search and the certification, and drafted this manuscript. Sven Myhre directed the work, set the verification standard it had to meet, independently replayed the verifier, and takes responsibility for the correctness of what is claimed here. An external reviewer's critique of an earlier draft prompted the oracle cross-check of the $h = 10$ machine and the controls described in Section 4; we thank them.

References

[1] V. Chvátal and D. Sankoff. Longest common subsequences of two random sequences. *Journal of Applied Probability*, 12(2):306–315, 1975.

[2] J. M. Steele. An Efron–Stein inequality for nonsymmetric statistics. *Annals of Statistics*, 14(2):753–758, 1986.

[3] J. Boutet de Monvel. Extensive simulations for longest common subsequences. *European Physical Journal B*, 7:293–308, 1999.

[4] G. S. Lueker. Improved bounds on the average length of longest common subsequences. *Journal of the ACM*, 56(3):1–38, 2009.

[5] G. T. Heineman, C. Miller, D. Reichman, A. Salls, G. N. Sárközy, and D. Soiffer. Improved lower bounds on the expected length of longest common subsequences. arXiv:2407.10925, 2024.